Lindel

Many dimensions. One sortable integer.

Install & load

```
INSTALL lindel FROM community;
LOAD lindel;
```

01 The four functions

Encode a fixed-size numeric array into an unsigned integer; decode it with the matching curve.

Function	Purpose
`hilbert_encode(a)`	Stronger spatial locality
`morton_encode(a)`	Bit interleaving / Z-order
`hilbert_decode(k,n,f,u)`	Recover Hilbert inputs
`morton_decode(k,n,f,u)`	Recover Morton inputs

```
SELECT
  hilbert_encode([10, 20]::UINTEGER[2]) AS h,
  morton_encode([10, 20]::UINTEGER[2]) AS m;
-- h = 884, m = 408 (both UBIGINT)
```

Cast explicitly: `UINTEGER[2]` is an ARRAY; an uncast `[10, 20]` is a LIST.

02 Choose your input width

Element type	Dimensions
`TINYINT` / `UTINYINT`	1–16
`SMALLINT` / `USMALLINT`	1–8
`INTEGER` / `UINTEGER` / `FLOAT`	1–4
`BIGINT` / `UBIGINT` / `DOUBLE`	1–2

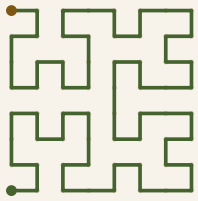
The output uses the smallest unsigned width that fits all component bits:

Input array	Output
`UTINYINT[2]`	`USMALLINT`
`UINTEGER[2]`	`UBIGINT`
`UINTEGER[3]`	`UHUGEINT`
`DOUBLE[2]`	`UHUGEINT`

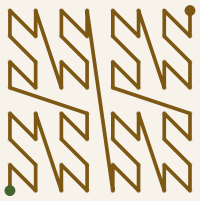
Prepare meaningful coordinates

For numeric locality across signed or floating values, shift and quantize to nonnegative integers. Direct encoding preserves their bit patterns; it does not normalize numeric distance. Keep precision and units consistent.

03 Choose a curve



Hilbert
Fewer long jumps; a good starting point for locality.



Morton / Z-order
Simpler to compute; jumps at quadrant boundaries.

04 Decode & round-trip

```
SELECT hilbert_decode(
  hilbert_encode([10, 20]::UINTEGER[2]),
  2, false, true
) AS point;
-- [10, 20]
```

n = dimensions; `f` = return floats; `u` = return unsigned integers. For signed integers use `false, false`; for floats use `true, false`. Preserve the key's original SQL type. Fixed v1.5 builds support signed 1-D round trips; older 6435106 rejects them.

05 Write a clustered Parquet

```
COPY (
  SELECT x, y
  FROM (VALUES (3, 5), (1, 1), (7, 0),
               (2, 6), (5, 3), (0, 4))
       AS points(x, y)
  ORDER BY
    hilbert_encode([x, y]::UINTEGER[2])
) TO 'points.parquet' (FORMAT PARQUET);
```

Read using ordinary predicates on the original columns:

```
SELECT * FROM 'points.parquet'
WHERE x BETWEEN 1 AND 3
  AND y BETWEEN 1 AND 6;
```

The tiny fixture shows syntax. With many row groups, the ordering can improve min/max pruning; benchmark your data and filters. This is a file layout, not an index.

Full docs & examples ↗ Source ↗